

Deep Research: Avatrada 57 Topic 019

Engineering Report: Spread Capture Modeling for Transaction Cost Estimation

Date: October 26, 2023 **Status:** Final **Author:** Autonomous Technical Researcher

Executive Summary

This report provides a detailed engineering analysis of a Spread Capture Model for estimating transaction costs, as specified in the source documentation. The model simplifies transaction cost analysis (TCA) by assuming that the cost incurred is a direct fraction of the prevailing bid-ask spread, determined by the asset's liquidity. This document deconstructs the underlying mathematical principles, provides a concrete Python implementation strategy, and conducts a critical analysis of the model's limitations, potential failure modes, and avenues for enhancement. The core objective is to create a functional and well-understood pre-trade cost estimation tool.

1. Technical Deconstruction

The "Spread Capture Model" is a heuristic approach to pre-trade transaction cost estimation. It operates on the principle that a trader's execution price relative to the mid-price is a function of the bid-ask spread and the asset's liquidity.

1.1. Core Concepts

- **Bid-Ask Spread:** The difference between the highest price a buyer is willing to pay (bid) and the lowest price a seller is willing to accept (ask). This spread represents the theoretical maximum cost for a "round trip" (buy then immediately sell) and is the primary source of cost for impatient market takers.

- $\text{Spread Width (S_W)} = \text{Ask Price} - \text{Bid Price}$

- **Mid-Price:** The theoretical "fair" price, calculated as the average of the bid and ask.

- $\text{Mid-Price} = (\text{Ask Price} + \text{Bid Price}) / 2$

- **Spread Capture:** The portion of the spread a trader "pays" to execute a trade. An aggressive market order that immediately crosses the spread to find a counterparty captures 100% of the half-spread. A more patient limit order placed inside the spread might get filled with price improvement, thus capturing less than 100%.

1.2. Mathematical Formulation

The model estimates the realized cost per share/contract by applying a liquidity-dependent multiplier to the spread. The fundamental formula is:

```
// Mathematical Formula for Realized Spread Cost per Share
```

$$C = M_L * S_W$$

Where: * C is the **Realized Spread Cost** per share. This is the estimated transaction cost. * S_W is the **Spread Width** ($\text{Ask} - \text{Bid}$). * M_L is the **Liquidity Multiplier**, a coefficient representing the fraction of the spread captured.

Based on the provided specification: * For **Liquid Assets** (e.g., SPY ETF): $M_L = 0.5$. This implies that, on average, a trader can execute at the mid-price, effectively paying half the spread. This is a common assumption for patient, well-

managed execution algorithms in liquid markets that can achieve price improvement. * For **Illiquid Assets** (e.g., illiquid options): $M_L = 1.0$. This implies the trader must be aggressive and cross the entire spread to find liquidity, paying the full **Ask** price to buy or receiving the full **Bid** price to sell.

Example Calculation: * **SPY (Liquid):** Bid = \$420.00, Ask = \$420.02. * $S_W = \$0.02$ * $M_L = 0.5$ * $C = 0.5$ * $\$0.02 = \0.01 **per share** * **Illiquid Option:** Bid = \$1.50, Ask = \$1.70. * $S_W = \$0.20$ * $M_L = 1.0$ * $C = 1.0$ * $\$0.20 = \0.20 **per contract**

2. Implementation Strategy

This section outlines a practical approach to implementing the spread capture model in Python, including liquidity categorization and a complete code example.

2.1. Liquidity Tier Categorization

The prompt requires categorizing `liquidity_tier` based on Average Daily Volume (ADV). This is a standard industry practice. The thresholds are heuristic and should be calibrated based on the specific universe of traded assets.

Here is a proposed tiering system for equities:

- **Tier 1: 'LIQUID':** Highly liquid assets, typically large-cap stocks and major ETFs.
 - **Criteria:** $ADV > 5,000,000$ shares.
 - **Multiplier (M_L):** 0.5
- **Tier 2: 'SEMI-LIQUID':** Mid-cap stocks or less-traded ETFs.
 - **Criteria:** $500,000 < ADV \leq 5,000,000$ shares.
 - **Multiplier (M_L):** 0.75 (An intermediate value between the two extremes).
- **Tier 3: 'ILLIQUID':** Small-cap stocks, certain preferred shares, or thinly traded instruments.
 - **Criteria:** $ADV \leq 500,000$ shares.
 - **Multiplier (M_L):** 1.0

Note on Options: For options, ADV of contracts is a valid metric. However, **Open Interest** combined with volume provides a more robust liquidity signal. The `ILLIQUID` tier with `M_L = 1.0` is a safe starting assumption for most options outside of the most active front-month strikes on major indices (like SPY).

2.2. Python Implementation

We will use standard Python libraries. No special dependencies are required for the core logic.

2.2.1. Liquidity Tiering Function

First, a helper function to determine the liquidity tier from ADV.

```
def get_liquidity_tier(symbol: str, adv: float) -> str:
    """
    Categorizes a symbol into a liquidity tier based on its ADV.

    Args:
        symbol (str): The ticker symbol (for potential future logic).
        adv (float): The Average Daily Volume for the symbol.

    Returns:
        str: The liquidity tier ('LIQUID', 'SEMI-LIQUID', 'ILLIQUID').
    """
    if adv > 5_000_000:
        return 'LIQUID'
    elif 500_000 < adv <= 5_000_000:
        return 'SEMI-LIQUID'
    else:
        return 'ILLIQUID'
```

2.2.2. Core Cost Estimation Function

This function implements the mathematical model described in Section 1.

```
def estimate_cost(symbol: str, spread_width: float, liquidity_tier: str) ->
float:
```

```

"""
Estimates the transaction cost per share based on spread capture modeling.

Args:
    symbol (str): The ticker symbol being analyzed.
    spread_width (float): The current bid-ask spread width for the symbol.
    liquidity_tier (str): The liquidity category ('LIQUID', 'SEMI-LIQUID',
'ILLIQUID').

Returns:
    float: The expected transaction cost per share/contract.

Raises:
    ValueError: If spread_width is negative or liquidity_tier is unknown.
"""
if spread_width < 0:
    raise ValueError("Spread width cannot be negative.")

# Define the liquidity multipliers based on the tier
liquidity_multipliers = {
    'LIQUID': 0.5,
    'SEMI-LIQUID': 0.75, # Interpolated value for medium liquidity
    'ILLIQUID': 1.0
}

multiplier = liquidity_multipliers.get(liquidity_tier)

if multiplier is None:
    raise ValueError(f"Unknown liquidity_tier: '{liquidity_tier}'. "
                    f"Expected one of "
                    f"{list(liquidity_multipliers.keys())}")

# Calculate the estimated cost
realized_spread_cost = multiplier * spread_width

return realized_spread_cost

```

2.3. Example Workflow

Here is how the functions would be used in a hypothetical trading system.

```
# --- Hypothetical Market Data ---
market_data = {
    'SPY': {'adv': 75_000_000, 'spread': 0.01},
    'AAPL': {'adv': 55_000_000, 'spread': 0.01},
    'ZM': {'adv': 4_500_000, 'spread': 0.05},
    'PETS': {'adv': 150_000, 'spread': 0.08},
    'XYZ_OPTION': {'adv': 500, 'spread': 0.25} # Illiquid option
}

# --- Analysis Loop ---
for symbol, data in market_data.items():
    adv = data['adv']
    spread = data['spread']

    # 1. Determine liquidity tier
    # For options, we can override or use a different logic
    if 'OPTION' in symbol:
        tier = 'ILLIQUID'
    else:
        tier = get_liquidity_tier(symbol, adv)

    # 2. Estimate cost
    try:
        cost_per_share = estimate_cost(symbol, spread, tier)
        print(f"Symbol: {symbol:<10} | Tier: {tier:<12} | Spread: ${spread:.4f}
| Est. Cost/Share: ${cost_per_share:.4f}")
    except ValueError as e:
        print(f"Error processing {symbol}: {e}")
```

Expected Output:

```
Symbol: SPY          | Tier: LIQUID          | Spread: $0.0100 | Est. Cost/Share: $0.0050
Symbol: AAPL         | Tier: LIQUID          | Spread: $0.0100 | Est. Cost/Share: $0.
```

0050

Symbol: ZM | Tier: SEMI-LIQUID | Spread: \$0.0500 | Est. Cost/Share: \$0.

0375

Symbol: PETS | Tier: ILLIQUID | Spread: \$0.0800 | Est. Cost/Share: \$0.

0800

Symbol: XYZ_OPTION | Tier: ILLIQUID | Spread: \$0.2500 | Est. Cost/Share: \$0.

2500

3. Critical Analysis

While simple and effective for quick estimates, the Spread Capture Model has significant limitations. A rigorous engineering assessment must consider its failure modes, underlying assumptions, and potential for improvement.

3.1. Failure Modes & Edge Cases

- **Data Integrity:** The model is highly sensitive to the quality of input data.
 - *Stale Quotes:* Using a stale `spread_width` will produce a meaningless cost estimate.
 - *Crossed or Zero Spreads:* Market data feeds can occasionally show erroneous quotes where `Bid >= Ask`. The implementation correctly raises a `ValueError` for negative spreads, but a zero spread would result in a zero cost estimate, which is unrealistic. The system should flag such inputs as data quality issues.
- **Extreme Volatility:** During market stress (e.g., news events, flash crashes), spreads widen dramatically. The model will correctly predict a higher cost, but the static `M_L` multipliers may no longer hold. In a panic, even for liquid stocks, traders may be forced to cross the full spread (`M_L - > 1.0`) to guarantee execution.
- **Market Open/Close:** Spreads are typically wider and liquidity is lower at the market open and close. A single ADV value does not capture this intraday dynamic, leading to underestimation of costs during these periods.

3.2. Model Simplifications and Limitations

- **Ignores Market Impact:** This is the model's most critical limitation. It assumes the trader's order is small enough not to affect the market price. For large orders (relative to ADV or liquidity at the top of the book), the act of trading will move the price, creating an additional, often larger, cost known as "market impact" or "slippage." This model does **not** account for it.
- **Static Multipliers:** The multipliers (0.5, 1.0) are static heuristics. In reality, spread capture is a continuous variable influenced by:
 - **Execution Strategy:** An aggressive market order vs. a patient limit order.
 - **Order Size:** A larger order may have to "walk the book," paying a worse price.
 - **Venue:** Execution quality can differ between exchanges and dark pools.
- **Binary Liquidity View:** The tiering system is a simplification. Liquidity is a spectrum, not a set of discrete buckets. The transition from 5,000,000 ADV to 5,000,001 ADV should not cause a sudden jump in the cost multiplier.

3.3. Optimizations & Enhancements

- **Dynamic Liquidity Multiplier:** The `M_L` could be replaced with a continuous function `f(adv)` to provide a smoother cost curve. For example:
`M_L = 0.5 + 0.5 * exp(-k * ADV)`.
- **Incorporate Volatility:** The model could be enhanced to adjust the multiplier based on short-term volatility (e.g., using ATR - Average True Range). In high-volatility regimes, the multiplier should skew towards 1.0.
 - `C = M_L(adv, volatility) * S_W`
- **Machine Learning Approach:** For a more sophisticated solution, a regression model (e.g., Gradient Boosting, Neural Network) could be trained on historical trade data.
 - **Features:** `spread_width`, `adv`, `order_size_as_%_of_adv`, `volatility`, `time_of_day`, `book_depth`.

- **Target Variable:** The actual realized cost (slippage) from historical trades, measured as $(\text{Execution Price} - \text{Arrival Mid-Price})$.
- This approach would learn the complex, non-linear relationships between market conditions and transaction costs, implicitly modeling both spread capture and market impact.
- **Backtesting Framework:** Any transaction cost model, regardless of complexity, must be rigorously backtested. This involves comparing the model's pre-trade cost estimates against the actual execution costs achieved by the trading system over a historical period. This process is essential for validating and calibrating the model's parameters (e.g., ADV thresholds, multipliers).